TP1 – Fuga até o Camarim

Algoritmos I
Data de entrega: 25/09/2026

1 Objetivo do trabalho

O objetivo deste trabalho é modelar o problema computacional descrito a seguir utilizando estruturas de dados e algoritmos de grafos que permitam resolvê-lo de forma eficiente. Serão fornecidos alguns casos de teste e suas respectivas respostas, mas recomenda-se que cada aluno produza testes adicionais para validar sua implementação.

O código-fonte da solução e uma documentação sucinta, com no máximo cinco páginas, deverão ser submetidos via Moodle até a data limite indicada. A documentação deverá apresentar a modelagem do problema, justificar os algoritmos e estruturas de dados adotados e analisar a complexidade da solução.

2 Definição do problema

Após uma apresentação, um artista precisa atravessar os corredores de um centro de eventos para chegar ao camarim. Entretanto, um ou mais grupos de fãs estão espalhados pelo local e começam a procurar pelo artista. Ao encontrar novos corredores, os fãs avisam outras pessoas, de modo que a região congestionada se expande continuamente.

O local é representado por um mapa retangular dividido em células. Cada célula pode ser uma parede, um corredor livre, a posição inicial do artista, a posição inicial de um grupo de fãs ou o camarim. Movimentos são permitidos apenas entre células que compartilham um lado; portanto, não são permitidos movimentos diagonais.

O artista possui velocidade V, isto é, consegue atravessar V **células por segundo**, enquanto os fãs avançam **uma célula por segundo**. Assim, cada deslocamento do artista entre duas células adjacentes dura $1/V$ de segundo, e cada avanço dos fãs entre células adjacentes dura um segundo. Todos começam a se movimentar no instante zero, e o artista não pode permanecer parado.

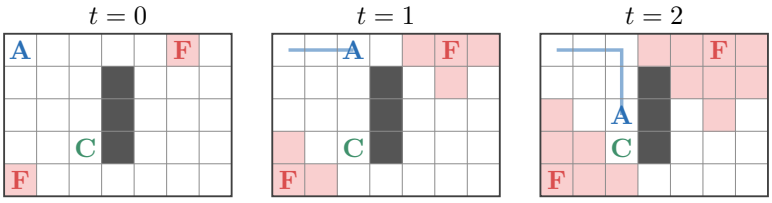


Figura 1: Três instantes da simulação para $V = 2$. As células em vermelho-claro já foram congestionadas; F indica a origem de um grupo de fãs, A indica a posição atual do artista e C indica o camarim. A linha azul mostra o percurso já realizado. Em cada segundo, o artista percorre duas células e, ao final, a região dos fãs avança uma célula.

Cada grupo de fã ocupa inicialmente uma célula distinta. A cada segundo, todos os grupos avançam simultaneamente para todas as células livres adjacentes. Uma célula alcançada por qualquer grupo torna-se congestionada e permanece assim até o final da simulação.

As paredes não podem ser atravessadas pelo artista nem pelos fãs. Uma célula alcançada pelos fãs fica congestionada até o fim da simulação. Uma colisão é contabilizada apenas na célula de chegada

de um movimento do artista: ele pode entrar em uma célula somente se chegar a ela antes dos fãs. Se ambos chegarem à célula no mesmo instante, o artista será abordado. Entretanto, se o artista passar por uma célula antes da chegada dos fãs, o movimento é válido, ainda que essa célula seja congestionada depois que ele a deixou. Por exemplo, se o artista passar por uma célula no instante $t = 4.5$ e os fãs a alcançarem no instante $t = 5$, não ocorre colisão. A Figura 1 ilustra três instantes em uma simulação.

A tarefa consiste em determinar se o artista consegue chegar ao camarim sem encontrar os fãs. Em caso positivo, devem ser informados o menor número de movimentos necessário e a quantidade de caminhos distintos que permitem chegar ao camarim com esse número mínimo. Dois caminhos são considerados distintos quando suas sequências de células visitadas são diferentes.

3 Formato da entrada

O programa deverá processar um único caso de teste em cada execução. A primeira linha contém três inteiros L, C e V, que representam, respectivamente, o número de linhas, o número de colunas do mapa e a velocidade do artista, com

$$2 \leq L, C \leq 100, \qquad 1 \leq V \leq 5 \qquad \text{e} \qquad L \cdot C \leq 10^4.$$

Cada uma das L linhas seguintes contém uma cadeia de exatamente C caracteres, sem espaços, descrevendo uma linha do mapa. Os caracteres possíveis são:

Caractere	Significado
`.`	corredor livre;
`#`	parede;
`A`	posição inicial do artista;
`F`	posição inicial de um grupo de fãs;
`C`	camarim.

O mapa contém exatamente um caractere `A`, exatamente um caractere `C` e pelo menos um caractere `F`. Pode-se assumir que todas as posições iniciais são distintas. Não se garante que exista um caminho seguro entre o artista e o camarim.

4 Formato da saída

Se não for possível chegar ao camarim sem encontrar os fãs, imprima:

```
-1 0
```

Caso contrário, imprima dois inteiros T e Q, separados por um espaço. O valor T representa o menor número de movimentos necessário para que o artista chegue ao camarim, e Q representa a quantidade de caminhos distintos que realizam o percurso em exatamente T movimentos, módulo $1\,000\,000\,007$. O tempo correspondente em segundos é T/V.

5 Exemplos

Exemplo 1

Entrada

```
7 9 1
#########
#A....F.#
#..C#...#
#...#...#
#...#...#
#.F.....#
#########
```

Saída

```
3 3
```

Com velocidade $V = 1$, cada movimento dura um segundo. O artista precisa de pelo menos três movimentos para chegar ao camarim. Existem três caminhos com esse tempo, e todos alcançam cada célula utilizada antes dos fãs.

Exemplo 2

Entrada

```
3 9 2
#########
#A..F..C#
#########
```

Saída

```
-1 0
```

Nesse caso, o único corredor entre o artista e o camarim é bloqueado pelos fãs, portanto não existe um caminho seguro.

Exemplo 3

Entrada

```
8 12 2
############
#A........##
#.....#....#
#..#..##.###
#...#......#
#.......#..#
#..FF...#.C#
############
```

Saída

```
14 6
```

O menor percurso seguro possui 14 movimentos, ou seja, sete segundos para $V = 2$. Há seis caminhos distintos que realizam o percurso nesse tempo mínimo.

Exemplo 4

Entrada

```
8 12 3
############
#A...#.#..F#
##.#.....#.#
#......#...#
#....#.....#
#...#......#
#.F...#...C#
############
```

Saída

```
14 39
```

Neste cenário, o artista chega ao camarim em 14 movimentos, correspondentes a 14/3 segundos. Existem 39 caminhos de tempo mínimo que respeitam a expansão dos dois grupos de fãs.

6 Implementação

O programa deverá ser implementado em C ou C++ utilizando apenas a biblioteca padrão da linguagem. O código deve compilar e executar nas máquinas do DCC. O arquivo de entrada será fornecido pela entrada padrão e o resultado deverá ser escrito na saída padrão, sem a criação de arquivos adicionais.

Para compilar a solução utilizando a estrutura disponibilizada, execute `make`. Ao término da compilação deverá ser produzido um executável chamado `tp01`, que poderá ser executado, por exemplo, com o comando:

```
./tp01 < casoTeste01.txt
```

7 O que deve ser entregue

Deverá ser submetido um arquivo `.zip` contendo apenas uma pasta chamada `tp1`, com os seguintes itens:

1. **Documentação em PDF:** relatório com no máximo cinco páginas contendo identificação (nome e matrícula), introdução, modelagem do mapa como grafo, descrição e justificativa dos algoritmos e estruturas de dados utilizados e análise de complexidade;
2. **Implementação:** arquivos-fonte e `Makefile` necessários para compilar o projeto.

A documentação deve explicar a solução do problema, e não apenas reproduzir ou comentar o código-fonte. Diagramas, pseudocódigo e pequenos exemplos podem ser utilizados quando ajudarem a apresentar a modelagem adotada.

7.1 Atrasos

Trabalhos poderão ser entregues após o prazo estabelecido, porém sujeitos a uma penalização regida pela seguinte fórmula:

$$\Delta_p = \frac{2^{(d-1)}}{0.32}\% \tag{1}$$

Nesta fórmula d representa dias de atraso. Por exemplo, se a nota dada pelo corretor for 70 e você entregou o TP com 4 dias corridos de atraso, sua penalização será de $\Delta_p = 25\%$ e, portanto, a sua nota final será: $N_f = 70(1 - \Delta_p) = 52.2$. Note que a penalização é exponencial e 6 dias de atraso resultam em uma penalização de 100%.

8 Considerações finais

Assim como em todos os trabalhos desta disciplina, é estritamente proibida a cópia parcial ou integral de códigos, seja da internet, LLMs, ou de colegas. As soluções poderão ser submetidas a ferramentas de detecção de similaridade. Além da correção dos resultados, serão consideradas a eficiência da solução, a clareza do código e a qualidade da documentação.